

AI Automation with n8n

Lesson 3: Basic Nodes and Data Flow

Basic Nodes and Data Flow - Study Notes

Key Concepts

1. **Trigger Node** – Starts a workflow when a specific event occurs.
2. **Set Node** – Creates, updates, or formats data fields for downstream use.
3. **HTTP Request Node** – Sends data to or retrieves data from external APIs/services.
4. **Data Flow** – The path data travels from trigger → transformation (Set) → external interaction (HTTP).
5. **Node Configuration Basics** – Mapping inputs/outputs, using expressions, and handling responses.

Summary

In a typical automation workflow, the **Trigger** node is the entry point. It listens for an event (e.g., a new record in a database, a webhook call, or a scheduled time) and provides the initial payload. Once the workflow is activated, the **Set** node lets you shape that payload: you can rename fields, combine values, add static data, or perform simple calculations. This step is crucial because downstream services often expect data in a particular format.

After the data is prepared, the **HTTP Request** node is used to communicate with external systems—sending the formatted payload to an API endpoint, retrieving information, or performing actions like creating a record in a SaaS app. Understanding how to map the Set node's output to the HTTP node's request body, headers, and query parameters ensures smooth integration. Together, these three nodes illustrate the core “receive → transform → deliver” pattern that underpins most low-code/automation platforms.

Important Points

- **Trigger Types:** Webhook, schedule (cron), database change, email receipt, etc. Choose the one that matches the event you need.
- **Set Node Tips:**
 - Use **expressions** (`{{ }}`) to reference previous node data (`{{ $json["field"] }}`).
 - Keep field names consistent with the target API's schema.
 - You can add static values (e.g., `"apiKey": "abcd1234"`).
- **HTTP Request Node Essentials:**
 - **Method:** GET, POST, PUT, PATCH, DELETE – pick based on the API contract.
 - **URL:** May contain path parameters; use expressions to inject dynamic values.
 - **Headers:** Commonly `Content-Type: application/json`` and authentication headers.
 - **Body:** Usually JSON; the Set node's output can be passed directly (`{{ $json }}`).
 - **Response Handling:** Store the response in a new variable or feed it into another node.
- **Error Handling:** Most platforms let you add “Error” branches; use them to log failures or retry the

HTTP request.

- **Testing:** Run the workflow with sample data in the Trigger node to verify each transformation before hitting a live API.

Examples

Example 1 – New Contact Notification

Goal: When a new contact is added to a CRM, send a welcome email via an external email service API.

1. **Trigger:** *CRM – New Contact* (provides `firstName`, `lastName`, `email`).

2. **Set:**

```
```json
{
 "to": "{{ $json.email }}",
 "subject": "Welcome, {{ $json.firstName }}!",
 "body": "Hi {{ $json.firstName }}, thanks for joining us."
}
```

- Creates the exact payload the email API expects.

3. **HTTP Request:**

- **Method:** POST
- **URL:** `https://api.emailservice.com/v1/send`
- **Headers:** `Authorization: Bearer {{ \$env.EMAIL\_API\_KEY }}`, `Content-Type: application/json`
- **Body:** `{{ \$json }}` (the output of the Set node)

**Result:** Each new CRM contact triggers an HTTP POST that sends a personalized welcome email.

---

### Example 2 – Weather Check & Slack Alert

**Goal:** Every morning, fetch the current weather and post a summary to a Slack channel.

1. **Trigger:** \*Schedule\* – every day at 07:00 AM.

2. **HTTP Request (Weather API):**

- **Method:** GET
- **URL:** `https://api.openweathermap.org/data/2.5/weather?q=London&appid={{ \$env.WEATHER\_KEY }}`
- **Headers:** none needed.
- **Response:** JSON containing `weather[0].description` and `main.temp`.

3. **Set:**

```
```json
{
  "text": "Good morning! Today's weather in London: {{ $json.weather[0].description }}, {{ ($json.main.temp - 273.15) | round(1) }}°C."
}
```

4. **HTTP Request (Slack):**

- **Method:** POST
- **URL:** `https://slack.com/api/chat.postMessage`
- **Headers:** `Authorization: Bearer {{ \$env.SLACK\_TOKEN }}`, `Content-Type: application/json`
- **Body:** `{{ \$json }}`

Result: At 7 /AM the workflow pulls the latest weather, formats a friendly message, and posts it to Slack.

Quick Review

1. What is the primary purpose of a **Trigger** node in a workflow?
2. How does the **Set** node help when working with an external API?
3. Which part of the **HTTP Request** node defines the type of operation you are performing (e.g., retrieve vs. create data)?
4. Name two ways to pass dynamic data from a previous node into the HTTP Request node.
5. Why is it important to handle errors after an HTTP Request node, and how can you do it?

Further Reading

- **Authentication Methods:** API keys, OAuth 2.0, JWT – how to store and use them securely.
- **Advanced Data Transformation:** Using Function/Code nodes for complex logic.
- **Rate Limiting & Retries:** Best practices for handling API throttling.
- **Conditional Branching:** Routing workflow paths based on HTTP response codes or data values.
- **Logging & Monitoring:** Capturing request/response logs for debugging and audit trails.

